SISTEM PEMESANAN TIKET EVENT KONSER

Nama	Muhammad Syahrul Pane
NIM	2410511090
Kelas	B
Dosen	Muhammad Panji Muslim, S.Pd., M.Kom
Semester	Genap TA. 2025/2026

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA
FAKULTAS ILMU KOMPUTER
S1 INFORMATIKA
2026

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web saat ini mendorong pergeseran paradigma dari aplikasi monolitik ke arsitektur microservice. Pendekatan monolitik menempatkan seluruh logika bisnis seperti autentikasi, manajemen event, pemrosesan pembayaran yang dibentuk dalam satu aplikasi tunggal. Hal ini menimbulkan masalah skalabilitas, ketergantungan antar modul yang tinggi, serta kesulitan dalam deployment dan pemeliharaan jangka panjang.

Sistem Pemesanan Tiket Event ini dirancang dengan pendekatan microservice untuk menjawab permasalahan tersebut. Sistem terdiri dari tiga service independen yang saling berkomunikasi melalui REST API, dilengkapi dengan API Gateway sebagai single entry point.

1.2 Tujuan

- Mengimplementasikan arsitektur microservice dengan minimal 3 service independen.
- Mengimplementasikan autentikasi berbasis JWT dengan access token dan refresh token.
- Mengimplementasikan login alternatif menggunakan Google OAuth 2.0 dengan Authorization Code Flow.
- Membangun salah satu service menggunakan PHP dengan pola MVC (Laravel 11).
- Mengimplementasikan API Gateway sebagai single entry point dengan JWT validation dan rate limiting.

1.3 Tujuan

Sistem mencakup fitur berikut:

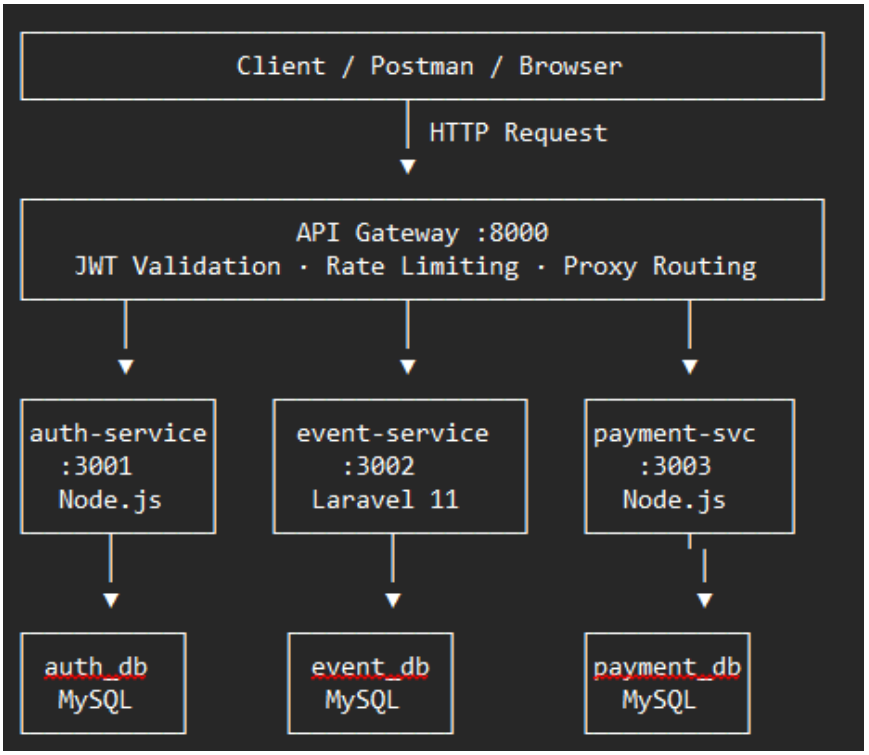
- Manajemen pengguna (registrasi, login, profil) dengan dukungan Google OAuth 2.0.
- Manajemen event dan kategori tiket.
- Pemesanan tiket (checkout, konfirmasi pembayaran, penerbitan e-ticket dengan QR code).
- Validasi QR code di pintu masuk event.
- API Gateway sebagai lapisan keamanan dan routing.

1.4 Tujuan

Komponen	Teknologi
auth-service	Node.js 18+, Express 4, MySQL
event-service	PHP 8.2, Laravel 11, MySQL
payment-service	Node.js 18+, Express 4, MySQL
API Gateway	Node.js 18+, Express 4, express-http-proxy
Autentikasi	JWT (jsonwebtoken), bcryptjs
OAuth	Google OAuth 2.0 Authorization Code Flow
Database	MySQL (3 database terpisah)
Version Control	Git, GitHub

2. ARSITEKTUR SISTEM

2.1 Gambaran Umum Arsitektur



2.2 Penjelasan Komponen

- API Gateway (port 8000) merupakan single entry point yang menerima semua request dari klien. Gateway bertanggung jawab atas validasi JWT, rate limiting global, dan meneruskan request ke service yang tepat berdasarkan prefix path.
- auth-service (port 3001) menangani semua urusan identitas pengguna: registrasi, login, penerbitan JWT, refresh token, logout dengan token blacklist, dan login via Google OAuth 2.0.
- event-service (port 3002) dibangun dengan Laravel 11 mengikuti pola MVC. Service ini mengelola data event, kategori tiket, kepemilikan tiket, dan validasi QR code di pintu masuk.
- payment-service (port 3003) mengelola proses checkout, konfirmasi pembayaran, dan berkomunikasi dengan event-service untuk menerbitkan e-ticket setelah pembayaran berhasil dikonfirmasi.

2.3 Alur Request

1. Klien mengirim request ke API Gateway `:8000`.
2. Gateway memeriksa apakah endpoint termasuk public route.
3. Jika bukan public route, Gateway memverifikasi JWT dari header `Authorization`.
4. Jika token valid, Gateway meneruskan request ke service yang sesuai berdasarkan prefix path.
5. Service downstream memproses request dan mengembalikan response.
6. Gateway meneruskan response kembali ke klien.

3. JUSTIFIKASI PEMISAHAN SERVICE

3.1 Mengapa Tidak Monolitik?

Pendekatan monolitik menempatkan seluruh kode (autentikasi, event, pembayaran) dalam satu aplikasi. Masalah yang muncul:

- Single point of failure: jika satu modul crash, seluruh aplikasi mati.
- Scaling tidak efisien: jika payment membutuhkan lebih banyak resource, kita harus scale seluruh aplikasi.
- Deployment coupled: perubahan kecil di satu modul memaksa deploy ulang keseluruhan.
- Tech lock-in: tidak bisa menggunakan teknologi berbeda per modul.

3.2 Justifikasi Pemisahan Per Service

- auth-service (dipisah karena: keamanan dan reusability)

  Autentikasi adalah lapisan keamanan yang digunakan oleh **semua** service lain. Memisahkannya memungkinkan:

  - JWT_SECRET dikelola di satu tempat, service lain hanya perlu tahu cara verifikasinya.
  - Pembaruan mekanisme autentikasi (misal: tambah MFA) tidak mempengaruhi logika bisnis service lain.
  - Dapat digunakan kembali oleh sistem lain di organisasi yang sama tanpa duplikasi kode.
  - event-service (dipisah karena: pola arsitektur berbeda dan domain boundary)

- event-service (dipisah karena: pola arsitektur berbeda dan domain boundary)

  Event-service sengaja dibangun dengan Laravel 11 (PHP MVC) karena:

  - Membuktikan bahwa microservice dapat menggunakan stack teknologi berbeda.
  - Domain "event dan tiket" memiliki boundary yang jelas dan tidak tumpang tindih dengan domain lain.
  - Memungkinkan tim backend PHP dan tim Node.js bekerja paralel tanpa konflik.

- payment-service (dipisah karena: kebutuhan konsistensi data dan skalabilitas berbeda)
  - Proses pembayaran memerlukan transaksi database yang ketat (BEGIN/COMMIT/ROLLBACK) yang sebaiknya diisolasi.
  - Beban traffic payment biasanya lebih tinggi di jam-jam tertentu (event besar), sehingga bisa di-scale secara independen.
  - Isolasi memudahkan integrasi dengan payment gateway eksternal (Midtrans, Xendit, dll.) di masa depan tanpa mengganggu service lain.

3.3 Perbandingan Monolitik vs Microservice

Aspek	Monolitik	Microservice (sistem ini)
Deployment	Satu aplikasi, semua atau tidak sama sekali	Tiap service deploy independen
Scaling	Scale seluruh aplikasi	Scale hanya service yang butuh

Database	Satu database, tabel shared	Tiga database terpisah, tidak ada shared table
Tech Stack	Satu bahasa/framework	Node.js + Laravel 11 (mixed)
Fault Isolation	Satu crash = semua mati	Crash di payment tidak mematikan auth/event
Komunikasi	Function call langsung	REST API via HTTP
Kompleksitas	Rendah di awal	Lebih tinggi, tapi lebih maintainable jangka panjang

4. PERANCANGAN DATABASE

Sistem menggunakan tiga database MySQL yang sepenuhnya terpisah. Tidak ada tabel yang diakses bersama antar service — komunikasi antar service dilakukan melalui REST API, bukan melalui join lintas database.

4.1 Database auth_db (auth-service)

Tabel users

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
name	VARCHAR(100)	Nama lengkap pengguna
email	VARCHAR(150) UNIQUE	Email unik, dipakai untuk login
password	VARCHAR(255) NULL	Hash bcrypt, NULL jika login via OAuth
role	ENUM('user','admin')	Hak akses pengguna
oauth_provider	VARCHAR(50) NULL	Nama provider OAuth (google, dll)
oauth_id	VARCHAR(255) NULL	ID unik dari provider OAuth
avatar	TEXT NULL	URL foto profil dari provider OAuth
created_at	TIMESTAMP	Waktu registrasi
updated_at	TIMESTAMP	Waktu update terakhir

Tabel users

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
user_id	CHAR(36) FK → users.id	Pemilik token
token	TEXT	String refresh token
expires_at	DATETIME	Waktu kadaluarsa (maks 7 hari)
created_at	TIMESTAMP	Waktu pembuatan

Tabel token_blacklist

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
token	TEXT	Access token yang di-invalidate saat logout
expires_at	DATETIME	Waktu kadaluarsa token (untuk auto-clean)

created_at	TIMESTAMP	Waktu logout

4.2 Database auth_db (auth-service)

Tabel users

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
name	VARCHAR(100)	Nama lengkap pengguna
email	VARCHAR(150) UNIQUE	Email unik, dipakai untuk login
password	VARCHAR(255) NULL	Hash bcrypt, NULL jika login via OAuth
role	ENUM('user','admin')	Hak akses pengguna
oauth_provider	VARCHAR(50) NULL	Nama provider OAuth (google, dll)
oauth_id	VARCHAR(255) NULL	ID unik dari provider OAuth
avatar	TEXT NULL	URL foto profil dari provider OAuth
created_at	TIMESTAMP	Waktu registrasi
updated_at	TIMESTAMP	Waktu update terakhir

Tabel ticket_categories

Kolom	**Tipe**	**Keterangan**
id	CHAR(36) PK	UUID v4
event_id	CHAR(36) FK → events.id	Event pemilik kategori
name	VARCHAR(100)	Nama kategori (VIP, Regular, dll)
price	DECIMAL(12,2)	Harga per tiket
quota	INT	Total kuota yang tersedia
sold	INT DEFAULT 0	Jumlah tiket terjual
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

Tabel tickets

Kolom	**Tipe**	**Keterangan**
id	CHAR(36) PK	UUID v4
event_id	CHAR(36) FK → events.id	Event yang dihadiri

ticket_category_id	CHAR(36) FK → ticket_categories.id	Kategori tiket
owner_id	VARCHAR(36)	ID user dari auth-service
order_id	VARCHAR(36) NULL	ID order dari payment-service
qr_code	VARCHAR(36) UNIQUE	Token UUID untuk scan QR
status	ENUM('active','used','cancelled')	Status tiket
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

Tabel ticket_validations

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
ticket_id	CHAR(36) FK → tickets.id	Tiket yang divalidasi
validated_by	VARCHAR(36)	ID petugas dari auth-service
validated_at	TIMESTAMP	Waktu validasi
gate	VARCHAR(50) NULL	Nama gate (Gate A, Gate B, dll)
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

4.3 Database payment_db (payment-service)

Tabel orders

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
user_id	CHAR(36)	ID user dari auth-service
event_id	CHAR(36)	ID event dari event-service
total_amount	DECIMAL(14,2)	Total harga semua item
status	ENUM('pending','paid','cancelled','refunded')	Status pembayaran
payment_method	VARCHAR(50) NULL	Metode bayar (transfer, qris, dll)
paid_at	DATETIME NULL	Waktu konfirmasi pembayaran
expired_at	DATETIME	Order kadaluarsa jika tidak dibayar dalam 15 menit
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

Tabel orders_items

Kolom	Tipe	Keterangan
id	CHAR(36) PK	UUID v4
order_id	CHAR(36) FK → orders.id	Order induk
ticket_category_id	CHAR(36)	ID kategori dari event-service
ticket_category_name	VARCHAR(100)	Nama kategori (snapshot saat checkout)
quantity	INT	Jumlah tiket dipesan
unit_price	DECIMAL(12,2)	Harga per tiket saat checkout
subtotal	DECIMAL(14,2)	quantity × unit_price
created_at	TIMESTAMP	—

4.4 Relasi Logis Antar Service

Karena setiap service memiliki database terpisah, foreign key lintas service tidak dapat diterapkan secara langsung di database. Referensi antar service disimpan sebagai VARCHAR dan divalidasi melalui REST API call.

Field	Tabel	Merujuk ke
organizer_id	events	users.id (auth_db)
owner_id	tickets	users.id (auth_db)
validated_by	ticket_validations	users.id (auth_db)
user_id	orders	users.id (auth_db)
event_id	orders	events.id (event_db)

5. REST API -HTTP METHOD & RESPONSE CODE

Sistem menerapkan HTTP method secara semantik sesuai standar REST:

Method	Semantik	Contoh Penggunaan
GET	Ambil data, tidak mengubah state	GET /api/events, GET /api/orders/:id
POST	Buat resource baru	POST /api/auth/register, POST /api/orders
PUT	Update resource secara penuh	PUT /api/events/:id
PATCH	Update sebagian field	(opsional, dipakai di update profil)
DELETE	Hapus resource	DELETE /api/orders/:id

HTTP Response Code yang Diterapkan

Code	Makna	Contoh Penggunaan
200 OK	Request berhasil, ada response body	GET /api/auth/me, POST /api/auth/login
201 Created	Resource baru berhasil dibuat	POST /api/auth/register, POST /api/events
204 No Content	Berhasil, tanpa response body	DELETE /api/events/:id
400 Bad Request	Request malformed	Body JSON tidak valid
401 Unauthorized	Token tidak ada, tidak valid, atau kadaluarsa	Akses endpoint terproteksi tanpa token
403 Forbidden	Token valid tapi tidak punya izin	User biasa mencoba hapus event milik orang lain
404 Not Found	Resource tidak ditemukan	GET /api/events/:id dengan id tidak ada
409 Conflict	Konflik state	Register dengan email yang sudah terdaftar
422 Unprocessable Entity	Validasi input gagal	Password kurang dari 8 karakter
500 Internal Server Error	Error tidak terduga di sisi server	Database connection error

6. IMPLEMENTASI JWT AUTHENTICATION

6.1 Alur Autentijasi JWT

1. User POST /api/auth/login dengan email + password
2. auth-service verifikasi password menggunakan bcrypt.compare()
3. Jika cocok, generate dua token:
   - Access Token : expires 15 menit (JWT_EXPIRES_IN=15m)
   - Refresh Token : expires 7 hari (REFRESH_TOKEN_EXPIRES_IN=7d)
4. Refresh token disimpan di tabel refresh_tokens dengan expires_at
5. Kedua token dikembalikan ke klien
6. Klien menyertakan Access Token di header: Authorization: Bearer <token>
7. Setiap request ke endpoint terproteksi melalui jwtMiddleware:
   a. Cek apakah token ada di blacklist (tabel token_blacklist)
   b. Verifikasi signature + expiry menggunakan jwt.verify()
   c. Jika valid, inject req.user = decoded payload
8. Saat access token kadaluarsa, klien POST /api/auth/refresh
   dengan refresh_token terima pasangan token baru (token rotation)
9. Saat logout, POST /api/auth/logout:
   a. Access token dimasukkan ke token_blacklist
   b. Refresh token dihapus dari tabel refresh_tokens

6.2 Spesifikasi Token

Parameter	Access Token	Refresh Token
Secret	JWT_SECRET	REFRESH_TOKEN_SECRET
Expires	15 menit	7 hari
Payload	id, email, role, iat, exp	id, email, role, iat, exp
Penyimpanan	Header Authorization klien	Tabel refresh_tokens di DB
Invalidasi	Token blacklist	Hapus dari DB saat logout/rotation

6.3 Token Rotation

Setiap kali refresh token digunakan untuk mendapatkan access token baru, refresh token lama dihapus dari database dan refresh token baru diterbitkan. Hal ini mencegah penggunaan ulang refresh token yang dicuri.

6.3 Token Rotation

Method	Path	Auth	Keterangan
POST	/api/auth/register	X	Registrasi akun baru
POST	/api/auth/login	X	Login, terima access + refresh token
POST	/api/auth/refresh	X	Tukar refresh token

			→ token baru
POST	/api/auth/logout	JWT	Blacklist access token + hapus refresh token
GET	/api/auth/me	JWT	Profil user yang sedang login

7. IMPLEMENTASI OAUTH 2.0 GOOGLE

7.1 Provider

Berdasarkan paritas NIM 2410511090 (digit terakhir: 0), provider OAuth yang digunakan adalah Google.

7.2 Flow yang Digunakan: Authorization Code Flow

Sistem menggunakan Authorization Code Flow (bukan Implicit Flow yang sudah deprecated) sesuai dengan RFC 6749.

7.3 Alur Authorization Code Flow

1. User klik "Login dengan Google"
   Browser diarahkan ke GET /api/auth/oauth/google

2. auth-service redirect browser ke Google Consent Screen:
   https://accounts.google.com/o/oauth2/v2/auth
   ?client_id=...
   &redirect_uri=http://localhost:3001/api/auth/oauth/google/callback
   &response_type=code ← Authorization Code (bukan token)
   &scope=openid email profile
   &access_type=offline

3. User login di Google dan klik "Allow"

4. Google redirect ke:
   GET /api/auth/oauth/google/callback?code=AUTHORIZATION_CODE

5. auth-service tukar authorization code dengan access token Google:
   POST https://oauth2.googleapis.com/token
   { code, client_id, client_secret, redirect_uri, grant_type: 'authorization_code' }

6. Menggunakan Google Access Token, ambil profil user:
   GET https://www.googleapis.com/oauth2/v3/userinfo
   → { sub, email, name, picture }

7. Mapping ke user lokal:
   a. Cari user berdasarkan oauth_provider='google' dan oauth_id=sub
   b. Jika tidak ada, cari berdasarkan email
      - Jika email ada → link akun OAuth ke user existing
      - Jika tidak ada → buat user baru dengan oauth_provider='google'
   c. Simpan nama, email, dan foto profil (avatar) dari Google

8. Generate JWT lokal (access + refresh token) → kembalikan ke klien

7.4 Data yang Disimpan dari Google

Field	Sumber	Disimpan di
name	Google profile (name)	users.name
email	Google profile (email)	users.email

foto profil	Google profile (picture)	users.avatar
oauth_provider	Hardcode 'google'	users.oauth_provider
oauth_id	Google profile (sub)	users.oauth_id

7.5 Endpoint OAuth

Method	Path	Keterangan
GET	/api/auth/oauth/google	Redirect ke halaman consent Google
GET	/api/auth/oauth/google/callback	Callback, terima code, buat JWT lokal

8. API GATEWAY

8.1 Fungsi Gateway

API Gateway berjalan di port 8000 dan berfungsi sebagai:

1. Single entry point — klien hanya berkomunikasi dengan gateway, tidak langsung ke service.
2. JWT Validation — semua request ke protected route diverifikasi JWT-nya di gateway sebelum diteruskan.
3. Rate Limiting — membatasi request untuk mencegah abuse dan brute-force.
4. Proxy Routing — meneruskan request ke service yang tepat berdasarkan prefix path.

8.2 Peta Routing Gateway

Path Masuk	Service Tujuan	Port	Catatan
/api/auth/*	auth-service	3001	Rate limit ketat untuk /login & /register
/api/events/*	event-service	3002	GET listing/detail: publik
/api/tickets/*	event-service	3002	Semua endpoint butuh JWT
/api/orders/*	payment-service	3003	Semua endpoint butuh JWT
/health	Gateway sendiri	8000	Menampilkan peta routing aktif

8.3 Public Routes (bypass JWT di Gateway)

Route berikut dapat diakses tanpa token:

Method	Path
POST	/api/auth/register
POST	/api/auth/login
POST	/api/auth/refresh
GET	/api/auth/oauth/google
GET	/api/auth/oauth/google/callback
GET	/api/events (listing publik)
GET	/api/events/:id (detail publik)
GET	/health

8.4 Rate Limiting

Rule	Batas	Diterapkan Pada
Global	60 request/menit per IP	Semua endpoint
Auth ketat	10 request/menit per IP	/api/auth/login dan /api/auth/register

8.5 Cara Kerja JWT Validation di Gateway

Gateway memverifikasi JWT menggunakan JWT_SECRET yang sama dengan auth-service. Jika token valid, request diteruskan ke downstream service. Downstream service tidak perlu memverifikasi JWT lagi dari sisi keamanan gateway — namun pada implementasi ini, event-service dan payment-service tetap memverifikasi token untuk mendapatkan informasi user (req.user) yang diperlukan oleh controller.

9. IMPLEMENTASI PHP MVC — EVENT-SERVICE

9.1 Framework

event-service dibangun menggunakan Laravel 11 yang mengikuti pola arsitektur MVC (Model-View-Controller). Karena ini adalah REST API, layer View tidak digunakan — sebagai gantinya, Controller langsung mengembalikan response JSON.

9.2 Pemisahan Layer

Layer	File	Tanggung Jawab
Model	app/Models/Event.php	Definisi tabel, relasi, mass assignment
Model	app/Models/TicketCategory.php	Relasi ke Event dan Ticket
Model	app/Models/Ticket.php	Auto-generate QR code via boot()
Model	app/Models/TicketValidation.php	Log validasi tiket
Controller	app/Http/Controllers/EventController.php	CRUD event + paging + filtering
Controller	app/Http/Controllers/TicketController.php	CRUD tiket + validasi QR
Middleware	app/Http/Middleware/JwtMiddleware.php	Verifikasi JWT dari auth-service
Route	routes/api.php	Definisi semua endpoint API

Tidak ada logika bisnis yang ditulis di dalam file route — semuanya didelegasikan ke Controller.

9.3 Fitur Paging dan Filtering

Endpoint GET /api/events mendukung paging dan filtering:
GET
/api/events?status=published&search=konser&date_from=2026-07-01&page=1&per_page=10

Parameter	Tipe	Keterangan
status	string	Filter berdasarkan status event
search	string	Pencarian judul event (LIKE %keyword%)
date_from	date	Filter event mulai tanggal ini
date_to	date	Filter event sampai tanggal ini
page	integer	Nomor halaman (default: 1)
per_page	integer	Jumlah item per halaman (default: 10, max: 100)

Response paging:

```
{
  "success": true,
  "data": [...],
  "meta": {
    "current_page": 1,
    "per_page": 10,
    "total": 45,
    "last_page": 5
  }
}
```

9.4 Komunikasi Inter-Service (event-service dikonsumsi service lain)

event-service tidak berdiri sendiri — payment-service mengonsumsinya:

- Saat checkout: payment-service memanggil GET /api/events/:id untuk validasi event dan kuota tiket.
- Setelah pembayaran: payment-service memanggil POST /api/tickets untuk menerbitkan e-ticket.

10. KOMUNIKASI ANTAR SERVICE

10.1 Alur Checkout dan Penerbitan Tiket

```
Client → Gateway → payment-service
                        |
                        ├─ GET /api/events/:id → event-service
                        |  (validasi event, cek kuota, ambil harga)
                        |
                        ├─ Buat order di payment_db
                        |
                        └─ [setelah konfirmasi bayar]
                           POST /api/tickets → event-service
                           (terbitkan e-ticket, kurangi kuota)
```

10.2 Format Inter-Service Call

payment-service memanggil event-service dengan menyertakan JWT yang sama dari klien:

```
await axios.post(`${EVENT_SERVICE_URL}/api/tickets`, {
  ticket_category_id: item.ticket_category_id,
  owner_id:           order.user_id,
  order_id:           order.id,
  quantity:           item.quantity,
}, {
  headers: {
    Authorization: req.headers['authorization'],  // teruskan JWT klien
  }
});
```

11. PENGUJIAN ENDPOINT

11.1 Tool Pengujian

Pengujian dilakukan menggunakan Postman dengan collection yang tersimpan di postman/collection.json.

11.2 Hasil Pengujian

Auth Service

Endpoint	Method	Status	Response
/api/auth/register	POST	201	User berhasil dibuat
/api/auth/login	POST	200	Access + refresh token
/api/auth/me	GET	200	Data profil user
/api/auth/refresh	POST	200	Token baru (rotation)
/api/auth/logout	POST	200	Token di-blacklist
/api/auth/oauth/google	GET	200	Redirect ke Google

Event Service

Endpoint	Method	Status	Response
/api/events	GET	200	Listing dengan paging
/api/events/:id	GET	200	Detail event + kategori
/api/events	POST	201	Event baru dibuat
/api/events/:id	PUT	200	Event diperbarui

Payment Service

Endpoint	Method	Status	Response
/api/orders	POST	201	Order dibuat, expired 15 menit
/api/orders	GET	200	Listing order dengan paging
/api/orders/:id	GET	200	Detail order + items
/api/orders/:id/confirm	POST	200	Bayar + e-ticket diterbitkan
/api/orders/:id	DELETE	200	Order dibatalkan

Tiket

Endpoint	Method	Status	Response
/api/tickets	GET	200	Daftar tiket milik user
/api/tickets/:id	GET	200	Detail tiket

/api/tickets/validate-qr	POST	200	QR valid, tiket jadi "used"

12. KESIMPULAN

Fitur yang Berhasil Diimplementasikan

1. Arsitektur Microservice — 3 service independen (auth, event, payment) + 1 API Gateway, masing-masing dengan database terpisah, berkomunikasi via REST API.
2. JWT Authentication — register, login, access token (15 menit), refresh token (7 hari) dengan rotation, logout dengan blacklist. Semua endpoint terproteksi menggunakan middleware JWT.
3. Google OAuth 2.0 — Authorization Code Flow, mapping ke user lokal, simpan nama, email, dan foto profil.
4. PHP MVC dengan Laravel 11 — event-service memisahkan layer Model, Controller, Route, dan Middleware. Dikonsumsi oleh payment-service melalui REST API.
5. Paging dan Filtering — endpoint GET /api/events mendukung parameter page, per_page, status, search, date_from, date_to.
6. API Gateway — single entry point, JWT validation di sisi gateway, rate limiting global (60 req/menit) dan ketat untuk endpoint auth (10 req/menit), peta routing ke semua service.
7. HTTP Semantics — penggunaan HTTP method (GET, POST, PUT, DELETE) dan response code (200, 201, 204, 400, 401, 403, 404, 409, 422, 500) yang sesuai standar REST.